

Pytorch and Scikit-learn

Machine Learning and Security

Fabio De Gaspari
Dorjan Hitaj

Scikit-learn

- An open-source Python library for machine learning and data mining
- Provides a consistent, well-designed API to implement a wide range of ML algorithms
- Simple, modular, and sufficiently performant
- Supports supervised learning (classification, regression), unsupervised learning (clustering, dimensionality reduction), and model selection

Scikit-learn: Estimators

Models interface based on *estimators*

- uniform interface implementing the *fit()* function, used to fit (train) the model on a dataset: *model.fit(Xs, Ys)*
- once fitted, the estimator can be used to predict target values of new data (classification or regression): *model.predict(X)* function

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> clf = RandomForestClassifier(random_state=0)
>>> X = [[ 1,  2,  3],  # 2 samples, 3 features
...      [11, 12, 13]]
>>> y = [0, 1]  # classes of each sample
>>> clf.fit(X, y)
RandomForestClassifier(random_state=0)
>>> clf.predict(X)  # predict classes of the training data
array([0, 1])
>>> clf.predict([[4, 5, 6], [14, 15, 16]])  # predict classes of new data
array([0, 1])
```

Scikit-learn: Transformations and Preprocessing

Preprocessing follows a similar interface to implement various transformations

- *fit()* function used to prime the transformation
  - *preprocessor.fit(X)* -> no Ys (targets/labels)
- once fitted, can preprocess using *preprocessor.transform(X)*
- Implements many transformations, including MinMaxScaler and StandardScaler (z-score)

```
>>> from sklearn.preprocessing import StandardScaler
>>> X = [[0, 15],
...      [1, -10]]
>>> # scale data according to computed scaling values
>>> StandardScaler().fit(X).transform(X)
array([[-1.,  1.],
       [ 1., -1.]])
```

Scikit-learn: Data and Datasets

Natively handles numpy arrays, but no support for pytorch Tensors

- Tensors can be turned into numpy arrays with *my_tensor.numpy()*
- datasets can be split into train/test with given percentage split (e.g., 80-20)
  - *x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)*
- provides functions to easily encode categorical features (e.g., colors)
  - *categorically_encoded = OrdinalEncoder().fit_transform(categorical_features)*
  - can encode multiple features at the same time, i.e., *categorical_features* can be a matrix
    - each column a different feature, each row a different sample

Scikit-learn: Performance Metrics

Implements many performance metrics with unified interface

- given a *fit()* Estimator:
- *estimator.score(x, y)*
  - returns accuracy for the fit model
- *metrics.precision_recall_fscore_support(y_true, y_pred, average=<val>)*
  - returns precision, recall, f1 score, and support for the fit model
  - <val> defines behavior if multiclass
  - support is the number of occurrences of each class in y_true.
  - f1 score: harmonic mean of precision and recall; balances both to measure in a single score
  - https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_recall_fscore_support.html

Scikit-learn: Quick Start

The standard pattern follows:

- model creation
- model fitting
- prediction and/or scoring

```
model = Estimator(**params)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
acc = model.score(X_test, y_test)
perf = precision_recall_fscore_support(y_test, y_pred, average='micro')
```

Scikit-learn: Examples of Estimators

Fit a Random Forest on a built-in dataset and test

- 3-class classification problem
- split dataset 80-20

```python
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

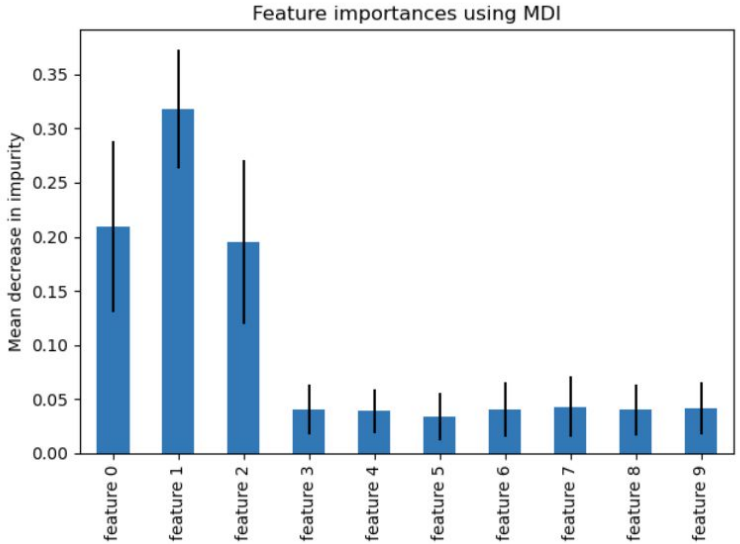
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
print("Test accuracy:", model.score(X_test, y_test))
```

Scikit-learn: Examples of Estimators

After fitting, *model.feature_importances_* returns the importance assigned by the tree to each feature

- mean and std of accumulation of the impurity decrease within each tree.



Scikit-learn: Data Frames

Two-dimensional, size-mutable, potentially heterogeneous tabular data

- Primary pandas data structure, contains labeled axes (rows and columns)

```
pandas.DataFrame(data=None, index=None, columns=None, dtype=None, copy=None)
```

- data: ndarray (structured or homogeneous), Iterable, dict, or DataFrame
- columns: column labels for DataFrame when data does not have them
  - defaulting to RangeIndex(0, 1, 2, …, n)
  - If data contains column labels, will perform column selection instead (e.g., dict)

Scikit-learn: Data Frames

DataFrame from dictionary

```
>>> d = {'col1': [1, 2], 'col2': [3, 4]}
>>> df = pd.DataFrame(data=d)
>>> df
   col1  col2
0     1     3
1     2     4
```

DataFrame from NumPy array

```
>>> df2 = pd.DataFrame(np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]),
...                    columns=['a', 'b', 'c'])
>>> df2
   a  b  c
0  1  2  3
1  4  5  6
2  7  8  9
```

- convert to numpy with *df.to_numpy()*

Scikit-learn: Data Frames

DataFrame from dictionary

```
train_df = pd.read_csv('path/train_data.csv')
test_df = pd.read_csv('path/test_data.csv')
train_df.info() # prints info about columns (features) and nubmer of rows (samples)
train_df.shape # retunrs shape of read data (rows, columns)
>>> (22544, 44)
```

you can index into the df using [column][index/slice], or use more complex conditions with *.loc[]* and *.iloc[]*

```
test_idxs = random.sample(range(dataset_df.shape[0]), 100)
y_test = dataset_df['label'][test_idxs]
x_test = dataset_df.loc[test_idxs, dataset_df.columns != 'label']
```

Scikit-learn: Data Frames and Scikit-learn

Integrates easily with scikit-learn built-ins

```
X = df.drop(columns=['label'])  # create a copy without targets/labels
y = df['label']

# Split the dataset into 80% training and 20% testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

and preprocessing to, e.g., encode categorical features and normalize data

```
categs_feats = train_df[['protocol_type', 'service', 'flag', 'attack']]
encoder = OrdinalEncoder()
categs_encoded = encoder.fit_transform(categs_feats)
train_df[['protocol_type', 'service', 'flag', 'attack']] = pd.DataFrame(categs_encoded, columns=categs_feats.columns)
scaler = MinMaxScaler()
x_train = scaler.fit_transform(train_df)
```

Scikit-learn: Data Frames and Scikit-learn

IMPORTANT! Fit your scaler on training data only, then apply the same fit scaler to test data to prevent biasing and information leakage!

- also, scalers and encoders return *NumPy arrays*

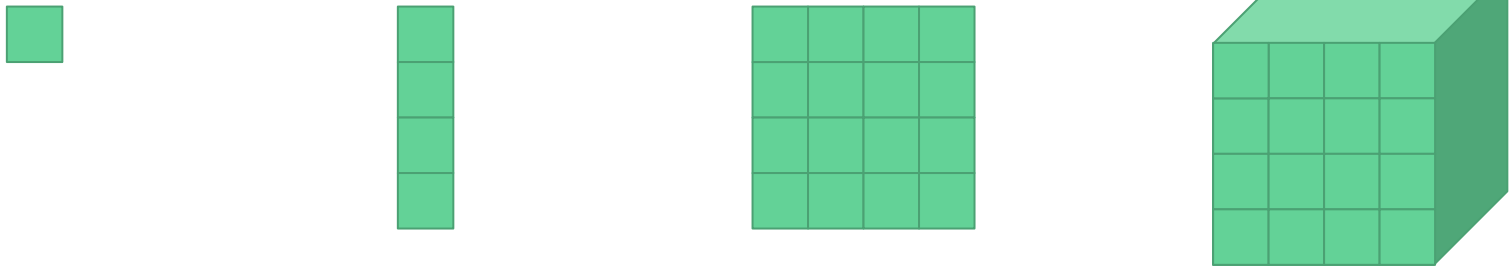
```
categs_feats = train_df[['protocol_type', 'service', 'flag', 'attack']]
encoder = OrdinalEncoder()
categs_encoded = encoder.fit_transform(categs_feats)
train_df[['protocol_type', 'service', 'flag', 'attack']] = pd.DataFrame(categs_encoded, columns=categs_feats.columns)
scaler = MinMaxScaler()
x_train = scaler.fit_transform(train_df)
```

PyTorch

Pytorch: Intro

Pytorch is based on the concept of *Tensors*

- you can think of tensors as a generalization of vector and matrices
  - a 1-dimensional tensor is equivalent to a vector
  - a 2-dimensional tensor is equivalent to a matrix
  - ...



Rank 0 Tensor (scalar)

Rank 1 Tensor (vector)

Rank 2 Tensor (matrix)

Rank 3 Tensor

Pytorch: Operations

Many ways to create Tensors, depending on goal:

- *torch.tensor(), torch.zeros(), torch.ones(), torch.randn(), etc.*

```
import torch
x1 = torch.tensor([1, 2, 3]) # tensor from list
x2 = torch.zeros(2, 4, 5) # zero-valued tensor of shape (2,4,5)
x3 = torch.ones(2, 3)  # one-valued tensor of shape (2,3)
x4 = torch.randn(2, 3)  # random real-valued tensor of shape (2,3)
```

Pytorch: Operations

Support all your typical operations: +, -, *, /

- also the strange @ operator, which performs matrix multiplication

Implements *broadcasting*

- given W, X and b

$$W = \begin{bmatrix} j & k & l \\ m & n & o \\ p & q & r \end{bmatrix} \quad X = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \quad b = \begin{bmatrix} s \\ t \\ u \end{bmatrix}$$

- Then WX + b:

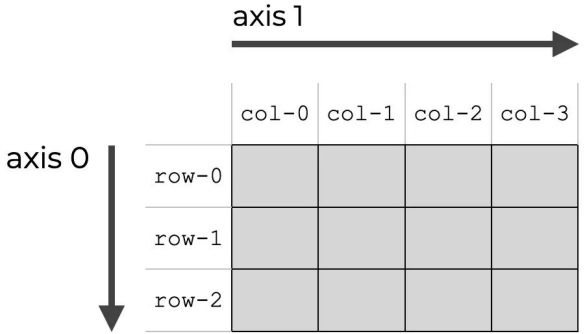
$$WX + b = \begin{bmatrix} (ja + kd + lg) + s & (jb + ke + lh) + s & (jc + kf + li) + s \\ (ma + nd + og) + t & (mb + ne + oh) + t & (mc + nf + oi) + t \\ (pa + qd + rg) + u & (pb + qe + rh) + u & (pc + qf + ri) + u \end{bmatrix}$$

Pytorch: Operations

Implements many axis-based functions

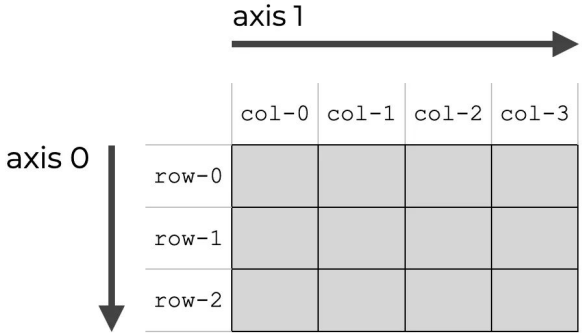
- functions that perform operations along a specific axis (dimension) of a tensor
- e.g., summing along rows/columns in a matrix, computing mean across certain dimensions

Axes (Dimensions): In a tensor, dimensions are referred to as axes. For example, Rank 2 (2D) tensor has two axes: axis 0 (rows) and axis 1 (columns).



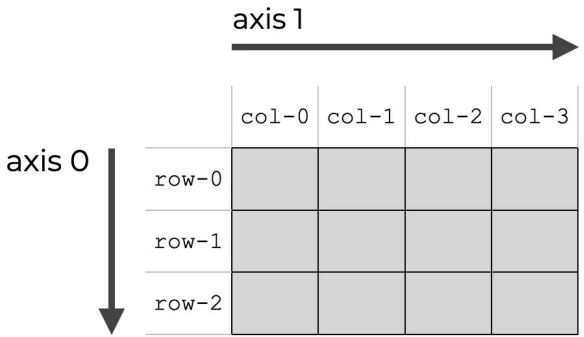
Pytorch: Operations

- torch.sum(): sum of all elements or along a specific axis.
- torch.mean(): cean of all elements or along a specific axis.
- torch.max() / torch.min(): maximum/minimum values along an axis.
- torch.prod(): product of all elements or along a specific axis.
- torch.std() / torch.var(): standard deviation and variance along an axis.



Pytorch: Operations

```
x1 = torch.Tensor([[1,2,3,4],[5,6,7,8]])
x1.shape
>>> torch.Size([2, 4])
x1.sum(axis=0)
>>> tensor([ 6.,  8., 10., 12.])
x1.sum(axis=1)
>>> tensor([10., 26.])
x1.sum()
>>> tensor(36.)
```



Pytorch: Operations

Some other useful functions

- x = torch.arange(0, 100, 10): same as python range()
- x.argmax(), x.argmin(): returns index of min/max element of tensor x
- x.item() returns a python number for a 1D tensor x (e.g. tensor [0.23] -> 0.23)
- Pytorch <--> NumPy transformations
  - x.numpy() returns a numpy array object from any arbitrary Tensor x
  - x = from_numpy(np_array) creates tensor from numpy array np_array
  - useful to interact with scikit-learn/pandas: turn dataframe into np array, then into tensor

see https://pytorch.org/docs/stable/tensors.html for more details

Pytorch: Tensor <--> NumPy

You can easily convert between Tensors and Numpy arrays

- torch.from_numpy(ndarray): returns a torch tensor from the np array
- torch.Tensor.numpy(): returns an ndarray from a torch tensor

Keep in mind that the original variable and the converted one *share the same underlying memory*

- changes in one reflect on the other
- if we convert a torch.Tensor to np.ndarray we typically have to call the .detach() function first: np_array = torch_tensor.detach().numpy()
  - we'll see soon why

Pytorch: Shapes

Functions to view/modify the shape of tensors

- tensor.shape: shows the shape of a tensor (# of elements for each dimension)
- tensor.ndim: rank of the tensor
- torch.reshape(tensor, shape): returns copy of tensor with new provided shape
- tensor.view(shape): provides a view of tensor in a different shape, without copy
- torch.stack(tensors, dim): stack tensors along given dimension
- torch.squeeze(tensor): removes dimensions with value 1 (same as np)
- indexing (x[1:4], x[:, 3], ...) works like python, with some more expressive power

Pytorch: Tensor vs NumPy ndarray

Why can’t we just use them interchangeably?

- they both represent n-dimensional data structures
- however, torch tensors have additional information related to pytorch *differentiation engine*

Pytorch Differentiation Engine: Autograd

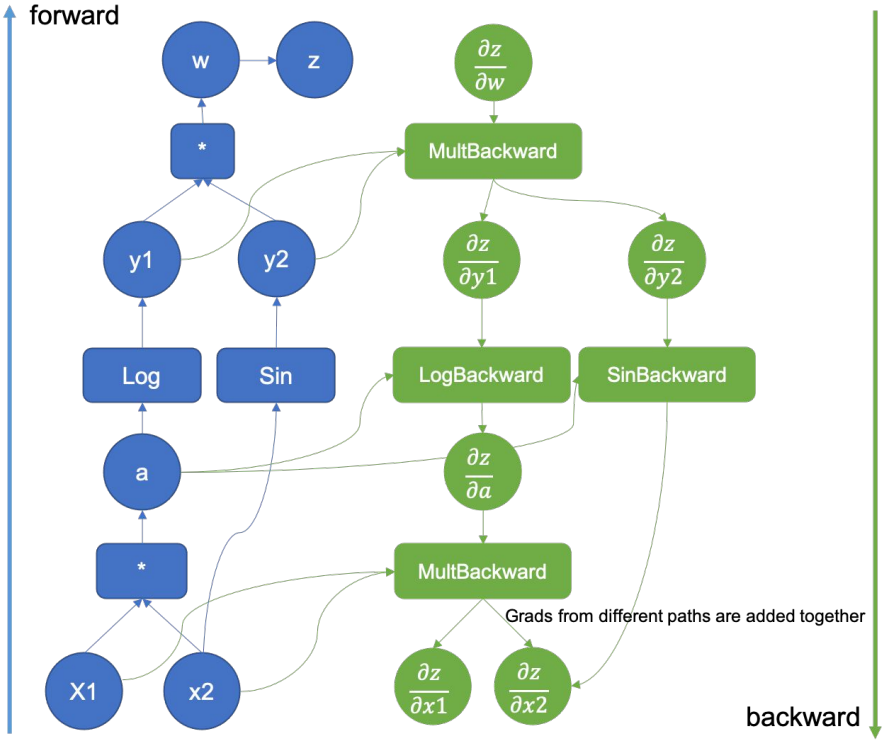
Pytorch uses a clever algorithm to automatically calculate gradients for NN

- each torch tensor has the *.requires_grad* flag, which indicates whether pytorch should calculate a gradient for it or not
  - remember that to train the model, we have to calculate the gradients wrt all weights and biases
  - weights and biases are Tensors in pytorch, and have .requires_grad = True by default

Pytorch Autograd engine is the component of the framework that calculates the derivatives wrt all the parameters of the networks

- it records all the operations performed on gradient-enabled tensors by creating a *computational graph*

Autograd: Computational Graph



from: pytorch documentation

Pytorch: Tensor vs ndarray

If we convert a torch.Tensor to np.ndarray we typically have to call the .detach() function first: np_array = torch_tensor.detach().numpy()

- the *detach()* function explicitly instructs the pytorch framework to remove from the computational graph the node associated with the torch_tensor variable

Since numpy arrays have no additional information required to create the computation graph, and since the tensor and nparray both share the same memory, we are required to detach the tensor from the graph, otherwise the auto-differentiation will break

DNN - From Scratch to Pytorch

DNN From Scratch With Pytorch

Single layer classification from scratch with pytorch

```
def log_softmax(x): # output layer activation
    return x - x.exp().sum(-1).log().unsqueeze(-1)

def negative_log_likelihood(input, target): # loss function
    return -input[range(target.shape[0]), target].mean()

class Mnist_Logistic_Scratch(nn.Module):
    def __init__(self):
        super().__init__()
        self.weights = torch.randn(784, 10) / math.sqrt(784)
        self.weights.requires_grad_()
        self.bias = torch.zeros(10, requires_grad=True)

    def forward(self, x):
        return log_softmax(x @ self.weights + self.bias)

    def pred(self, x):
        torch.argmax(self(x), dim=1)
```

DNN From Scratch With Pytorch

```
def fit(self, x_train, y_train, epochs, bs, lr):
    for epoch in range(epochs):
        for i in range((x_train.shape[0] - 1) // bs + 1):
            #         set_trace()
            start_i = i * bs
            end_i = start_i + bs
            x_batch = x_train[start_i:end_i]
            y_batch = y_train[start_i:end_i]
            pred = self(x_batch)
            loss = self.loss_fn(pred, y_batch)

            loss.backward()
            with torch.no_grad():
                self.weights -= self.weights.grad * lr
                self.bias -= self.bias.grad * lr
                self.weights.grad.zero_()
                self.bias.grad.zero_()
```

- loss.backward() automatically computes all the gradients for each parameter
- torch.no_grad() indicates that autograd should not record the following operations in the computation graph for the calculation of the next gradient

Pytorch: Essential Modules

Pytorch provides many modules to simplify common NN operations. The following four modules are essential in any kind of Pytorch project:

- torch.nn
- torch.optim
- torch.utils.data.Dataset
- torch.utils.data.Dataloader

Pytorch Essential Modules: torch.nn

torch.nn contains all the basic building blocks for the computational graph

- torch.nn.Module: base class for all NN modules. In pytorch, a NN is a Module, and its components are also modules
  - layers are implemented as nn.Module classes
  - activation functions are nn.Module classes
  - implements the *forward()* function, which tells the module what operations to compute on its input data
  - autograd generates the corresponding backward pass code automatically
- torch.nn.Parameter: a Tensor that is used as a NN parameter
  - e.g., weights and biases are torch.nn.Parameters
- torch.nn.Functional: implements many required functions
  - activations, loss, ...

DNN With Pytorch Functional and Parameter

Using torch.nn.Functional and torch.nn.Parameter to make things simpler

```python
def log_softmax(x): # output layer activation
    return x - x.exp().sum(-1).log().unsqueeze(-1)

def negative_log_likelihood(input, target): # loss function
    return -input[range(target.shape[0]), target].mean()

class Mnist_Logistic_Scratch(nn.Module):
    def __init__(self):
        super().__init__()
        self.weights = torch.randn(784, 10) / math.sqrt(784)
        self.weights.requires_grad_()
        self.bias = torch.zeros(10, requires_grad=True)

    def forward(self, x):
        return log_softmax(x @ self.weights + self.bias)

    def pred(self, x):
        torch.argmax(self(x), dim=1)
```



```python
import torch
import math
from torch import nn
import torch.nn.functional as F


class Mnist_Logistic(nn.Module):
    def __init__(self):
        super().__init__()
        self.weights = nn.Parameter(torch.randn(784, 10) / math.sqrt(784))
        self.bias = nn.Parameter(torch.zeros(10))
        self.loss_fn = F.cross_entropy

    def forward(self, x):
        return x @ self.weights + self.bias

    def pred(self, x):
        torch.argmax(self(x), dim=1)
```

DNN With Pytorch Functional and Parameter

```python
def fit(self, x_train, y_train, epochs, bs, lr):
    for epoch in range(epochs):
        for i in range((x_train.shape[0] - 1) // bs + 1):
            #         set_trace()
            start_i = i * bs
            end_i = start_i + bs
            x_batch = x_train[start_i:end_i]
            y_batch = y_train[start_i:end_i]
            pred = self(x_batch)
            loss = self.loss_fn(pred, y_batch)

            loss.backward()
            with torch.no_grad():
                self.weights -= self.weights.grad * lr
                self.bias -= self.bias.grad * lr
                self.weights.grad.zero_()
                self.bias.grad.zero_()
```



```python
def fit(self, x_train, y_train, epochs, bs, lr):
    for epoch in range(epochs):
        for i in range((x_train.shape[0] - 1) // bs + 1):
            start_i = i * bs
            end_i = start_i + bs
            x_batch = x_train[start_i:end_i]
            y_batch = y_train[start_i:end_i]
            pred = self(x_batch)
            loss = self.loss_fn(pred, y_batch)

            loss.backward()
            with torch.no_grad():
                for p in self.parameters():
                    p -= p.grad * lr
                self.zero_grad()
```

- when you call loss.backward(), autograd automatically stores the gradient for each torch.nn.Parameter of the model

DNN With Pytorch Layers

Using the nn.Linear module from Pytorch, instead of manually initializing weights and biases and computing the linear affine transformation

```
class Mnist_Logistic_Linear(nn.Module):
    def __init__(self):
        super().__init__()
        self.lin = nn.Linear(784, 10)
        self.loss_fn = F.cross_entropy

    def forward(self, x):
        return self.lin(x)

    def pred(self, x):
        torch.argmax(self(x), dim=1)
```

Pytorch has many types of built-in layers that can perform common operations for us

Pytorch Essential Modules: torch.optim

torch.optim contains various optimization algorithms

- Stochastic Gradient Descent is one of the potential optimization algorithms
- Adam optimizer is another alternative

Pytorch optimizers have a *step()* method used to automatically update the parameters with the chosen algorithm

- we don’t need to manually update all parameters with gradient and learning rate

DNN With Pytorch Functional and Parameter

Using torch.optimizer to abstract away the details of parameter optimization

```
def fit(self, x_train, y_train, epochs, bs, lr):
    for epoch in range(epochs):
        for i in range((x_train.shape[0] - 1) // bs + 1):
            start_i = i * bs
            end_i = start_i + bs
            x_batch = x_train[start_i:end_i]
            y_batch = y_train[start_i:end_i]
            pred = self(x_batch)
            loss = self.loss_fn(pred, y_batch)

            loss.backward()
            with torch.no_grad():
                for p in self.parameters():
                    p -= p.grad * lr
                self.zero_grad()
```



```
    # in class __init__ method
    self.optimizer = torch.optim.SGD(self.parameters(), lr=lr)

def fit(self, x_train, y_train, epochs, bs):
    for epoch in range(epochs):
        for i in range((x_train.shape[0] - 1) // bs + 1):
            start_i = i * bs
            end_i = start_i + bs
            x_batch = x_train[start_i:end_i]
            y_batch = y_train[start_i:end_i]
            pred = self(x_batch)
            loss = self.loss_fn(pred, y_batch)

            loss.backward()
            self.optimizer.step()
            self.optimizer.zero_grad()
```

Pytorch Essential Modules: Dataset and DataLoader

Pytorch provides useful classes to deal with dataset loading and iterations

- Dataset: abstract class implementing __len__ and __getitem__ functions
  - __len__ defines the length of the instance and __getitem__ allows indexing, similar to lists and np.array
- DataLoader: abstraction used to manage batches of the training set
  - makes it easier to handle batches, without having to manually create batches from a Dataset object

Pytorch Essential Modules: Dataset and DataLoader

Loading dataset into trainloader and testloader objects

```
import pickle
import gzip
from torch.utils.data import TensorDataset, DataLoader

def load_dataset():
    with gzip.open((PATH / FILENAME).as_posix(), "rb") as f:
        x_train, y_train, x_test, y_test = pickle.load(f)
        trainset = TensorDataset(x_train, y_train)
        testset = TensorDataset(x_test, y_test)
        trainloader = DataLoader(trainset, batch_size=128)
        testloader = DataLoader(testset, batch_size=128)
        return trainloader, testloader
```

Pytorch Essential Modules: Dataset and DataLoader

Using the DataLoader to easily iterate over mini batches

```
    # in class __init__ method
    self.optimizer = torch.optim.SGD(self.parameters(), lr=lr)

def fit(self, x_train, y_train, epochs, bs):
    for epoch in range(epochs):
        for i in range((x_train.shape[0] - 1) // bs + 1):
            start_i = i * bs
            end_i = start_i + bs
            x_batch = x_train[start_i:end_i]
            y_batch = y_train[start_i:end_i]
            pred = self(x_batch)
            loss = self.loss_fn(pred, y_batch)

            loss.backward()
            self.optimizer.step()
            self.optimizer.zero_grad()
```



```
def fit(self, trainloader, epochs):
    for epoch in range(epochs):
        for x_batch, y_batch in trainloader:
            pred = self(x_batch)
            loss = self.loss_func(pred, y_batch)

            loss.backward()
            self.optimizer.step()
            self.optimizer.zero_grad()
```

Pytorch: Deep Models

With Pytorch modules we can easily define a single layer (e.g., nn.Linear).

- We can use nn.Sequential() to chain together multiple layers and activations to create deep models

```
class Mnist_Deep_Linear(nn.Module):
    def __init__(self, lr):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(784, 20),
            nn.ReLU(),
            nn.Linear(20, 15),
            nn.ReLU(),
            nn.Linear(20, 10)
        )
        self.loss_fn = F.cross_entropy
        self.optimizer = torch.optim.SGD(self.parameters(), lr=lr)

    def forward(self, x):
        return self.model(x)
```

Pytorch: Train vs Test Mode

IMPORTANT: some modules are designed to work differently when training the models vs testing

- For example, batch normalization normalize the inputs of each layer in a neural network
  - standardizes the activations by scaling them to have a mean of 0 and variance of 1
  - BatchNormalization layers compute running mean and variance estimates during training
  - during testing, it uses training running mean/variance to perform the normalization

Before training, use *model.train()*. Before testing, user *model.eval()*

- they change the behavior of internal modules of *model* (e.g., BatchNorm modules) to perform the appropriate computations

Pytorch: Running Things on the GPU

you can check whether pytorch has access to GPU acceleration on your machine:

- *torch.cuda.is_available()*: true if pytorch detects an available GPU
- if false, you will have to run things on the CPU

You can move Tensors (and NN models) on a specific device by using .to:

- *my_tensor.to(device)*; *nn_model.to(device)*
- where *device* is the string “cuda” or “cpu”

You can move things back to cpu, for instance to conver to nparray:

- *my_array = my_tensor.cpu().detach().numpy()*

! model and data must be on the same device (cpu or cuda) !